

Duyệt cây & cây khung

Trần Vĩnh Đức - Nguyễn Lâm Tùng

Ngày 31 tháng 12 năm 2023

Nội dung

1 Duyệt cây

- Hệ địa chỉ phổ dụng
- Các thuật toán duyệt cây

2 Ký pháp trung tố, tiền tố, và hậu tố

3 Cây khung

- Mở đầu
- Tìm kiếm ưu tiên chiều sâu
- Tìm kiếm ưu tiên chiều rộng

Gán nhãn cho cây

- 1 *Gốc* có nhãn là 0
- 2 các *con của gốc* được gán nhãn *từ trái qua phải*:

1, 2, ... k

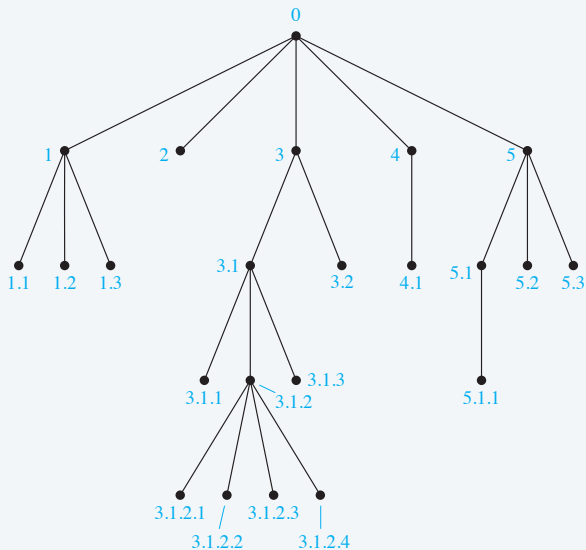
- 3 Với mỗi đỉnh v ở mức n có nhãn A thì các con của nó được gán nhãn *từ trái qua phải*:

A.1, A.2, ..., A.k

Thứ tự của các nhãn là thứ tự từ điển. Ví dụ

$$0 < 1 < 1.1 < 1.2 < 1.3 < 2 < 2.1 < 2.2 < 3 < 3.1$$

Hệ địa chỉ phổ dụng



Duyệt cây

Định nghĩa

Duyệt cây là đến thăm tất cả các đỉnh của cây hay nói cách khác là liệt kê tất cả các đỉnh của cây theo một thứ tự nào đó, mỗi đỉnh đúng một lần.

Có 3 phương pháp duyệt cây là:

- 1 Duyệt tiền thứ tự
- 2 Duyệt trung thứ tự
- 3 Duyệt hậu thứ tự

Duyệt tiền thứ tự

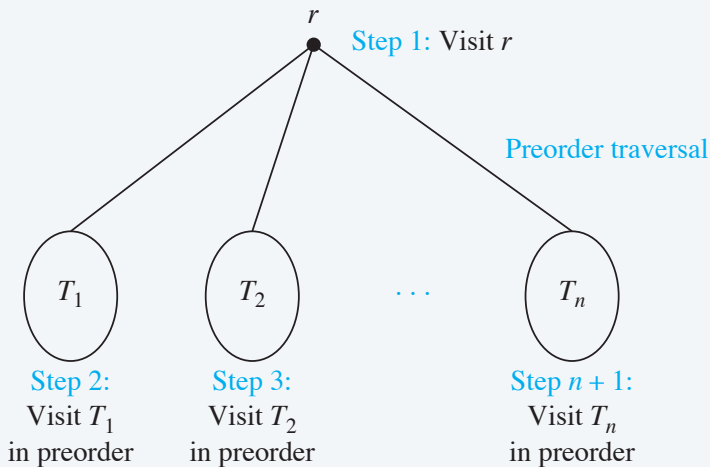
Định nghĩa

Giả sử T là một cây có gốc r .

- Nếu T chỉ có một đỉnh r thì r là cách *duyet tien thu tu* của T .
- Nếu $T_1, T_2, \dots, T_n$ là các cây con tại r từ trái qua phải:

Duyệt tiền thứ tự sẽ thăm r đầu tiên.

Tiếp tục duyệt T_1 theo *tiền thứ tự*, sau đó duyệt T_2 theo *tiền thứ tự*, cứ vậy cho đến khi T_n được duyệt theo kiểu *tiền thứ tự*.



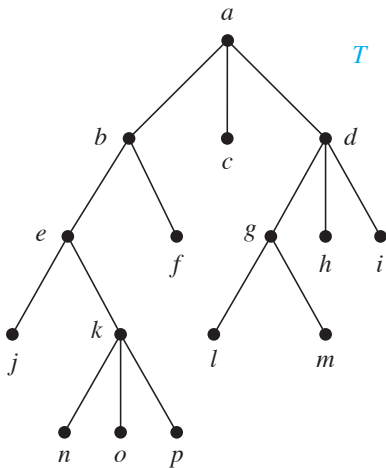
Hình: Duyệt tiền thứ tự

Thuật toán duyệt tiền thứ tự

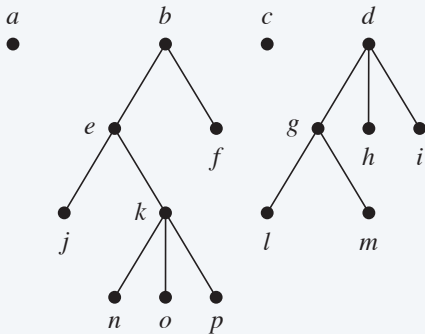
```
procedure preorder(Cây có gốc  $T$ )  
   $r =$  gốc của  $T$  ;  
  Thăm  $r$  ;  
  for mỗi con  $c$  của  $r$  tính từ trái qua phải do  
     $T(c) =$  cây con có gốc tại  $c$  ;  
    preorder ( $T(c)$ ) ;  
end
```


Ví dụ

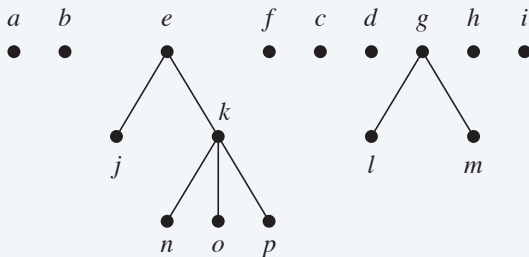
Hãy duyệt cây theo kiểu tiền thứ tự.



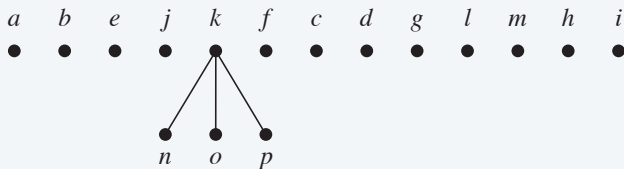
Ví dụ



Ví dụ



Ví dụ



Cuối cùng ta được:

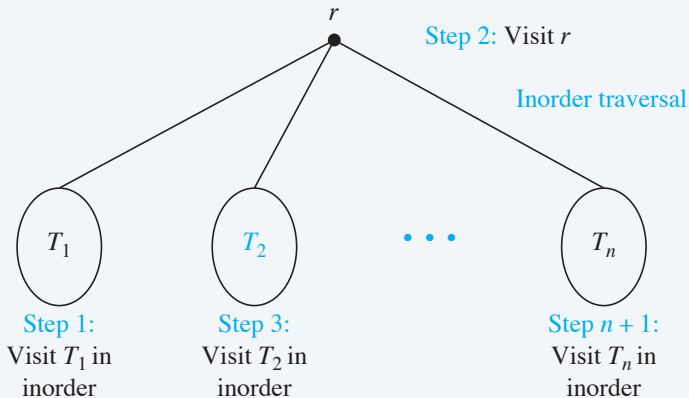
$a, b, e, j, k, n, o, p, f, c, d, g, l, m, h, i$

Duyệt trung thứ tự

Định nghĩa

Giả sử T là một cây có gốc r .

- Nếu T chỉ có một đỉnh r thì r là cách *duyet trung thứ tự* của T .
- Nếu $T_1, T_2, \dots, T_n$ là các cây con tại r từ trái qua phải:
Duyệt trung thứ tự sẽ bắt đầu bằng việc duyệt T_1 theo kiểu *trung thứ tự*,
sau đó viếng thăm r .
Tiếp tục duyệt T_2 theo *trung thứ tự*, tiếp tục duyệt T_3 theo kiểu *trung thứ tự*, cứ vậy cho đến khi T_n được duyệt theo kiểu *trung thứ tự*.



Hình: Duyệt trung thứ tự

Thuật toán duyệt trung thứ tự

procedure inorder(*Cây có gốc T*)

$r =$ gốc của T ;

if r là một lá **then**

 | Thăm r ;

end

else

$\ell =$ con đầu tiên của r tính từ trái qua phải;

$T(\ell) =$ cây con có gốc tại ℓ ;

 inorder ($T(\ell)$);

 Thăm r ;

for mỗi con $c \neq \ell$ của r tính từ trái qua phải **do**

 | $T(c) =$ cây con có gốc tại c ;

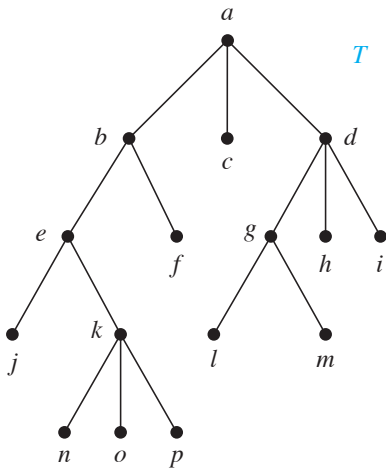
 | inorder ($T(c)$);

end

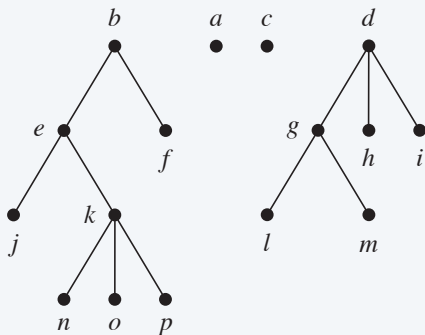
end

Ví dụ

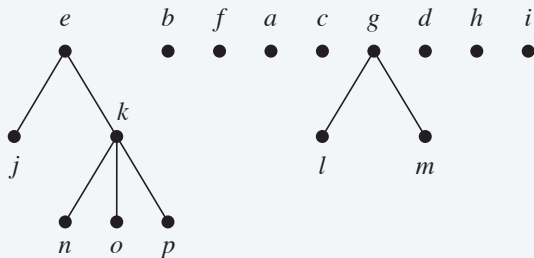
Hãy duyệt cây theo kiểu trung thứ tự.



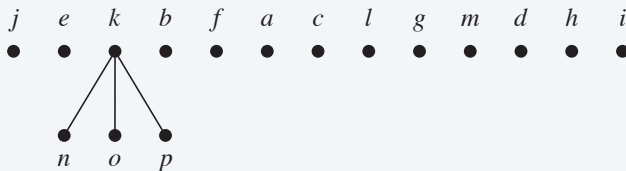
Ví dụ: Duyệt trung thứ tự



Ví dụ: Duyệt trung thứ tự



Ví dụ: Duyệt trung thứ tự



Cuối cùng ta được:

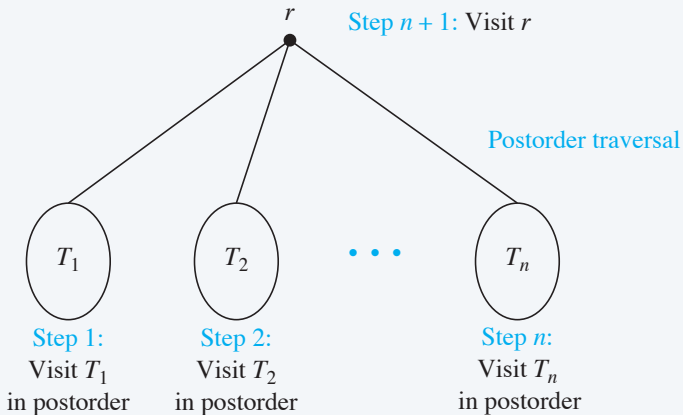
j, e, n, k, o, p, b, f, a, c, l, g, m, d, h, i

Duyệt hậu thứ tự

Định nghĩa

Giả sử T là một cây có gốc r .

- Nếu T chỉ có một đỉnh r thì r là cách *duyet hau thu tu* của T .
- Nếu $T_1, T_2, \dots, T_n$ là các cây con tại r từ trái qua phải:
Duyệt hậu thứ tự sẽ bắt đầu bằng việc duyệt T_1 theo kiểu *hau thu tu*, sau đó duyệt T_2 theo *hau thu tu*, và cứ tiếp tục cho đến khi T_n được duyệt theo kiểu *hau thu tu*,
và **kết thúc bằng việc thăm r** .



Hình: Duyệt hậu thứ tự

Thuật toán duyệt hậu thứ tự

procedure postorder(*Cây có gốc T*)

$r =$ gốc của T ;

for *mỗi con c của r tính từ trái qua phải* **do**

$T(c) =$ cây con có gốc tại c ;

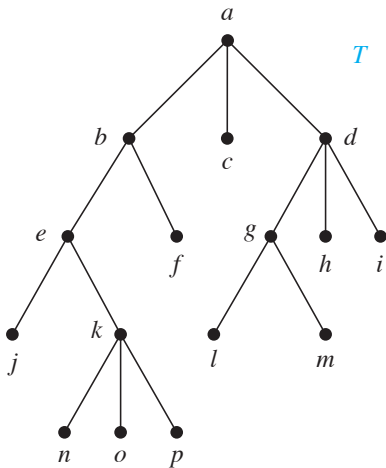
 postorder ($T(c)$);

end

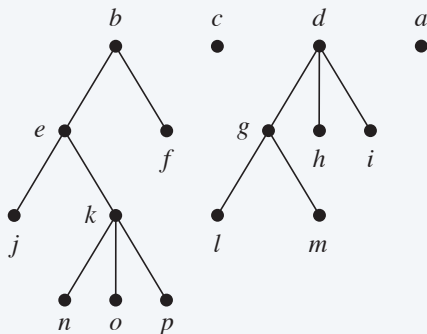
 Thăm r ;

Ví dụ

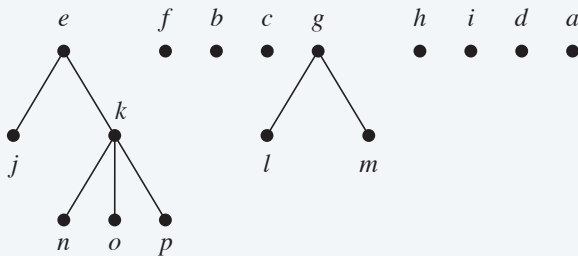
Hãy duyệt cây theo kiểu hậu thứ tự.



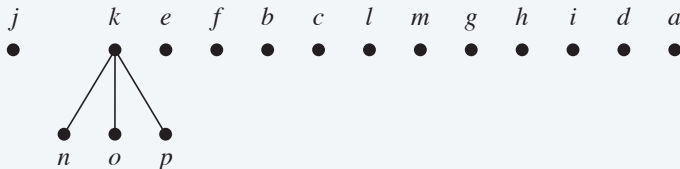
Ví dụ: Duyệt hậu thứ tự



Ví dụ: Duyệt hậu thứ tự



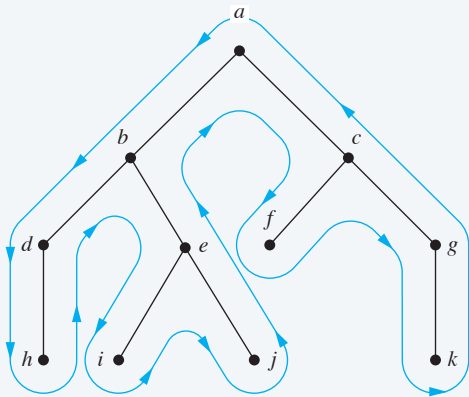
Ví dụ: Duyệt hậu thứ tự



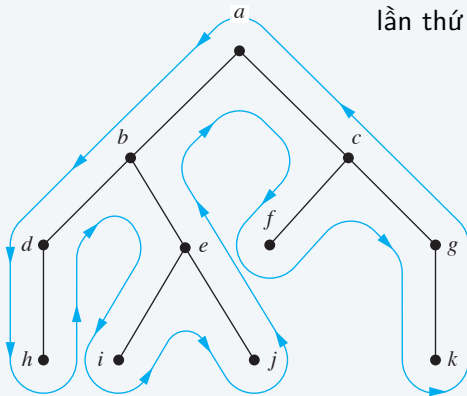
Cuối cùng ta được:

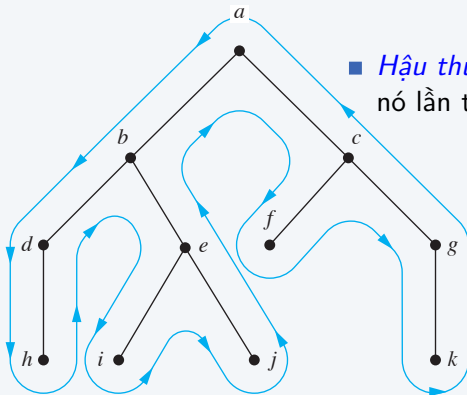
j, n, o, p, k, e, f, b, c, l, m, g, h, i, d, a

- *Tiền thứ tự*: Liệt kê mỗi đỉnh ngay khi đi qua nó



- *Trung thứ tự*: Liệt kê các lá khi đi qua lần đầu, liệt kê đỉnh trong khi đi qua lần thứ 2





- *Hậu thứ tự*: Liệt kê mỗi đỉnh khi đi qua nó lần trước lần trở về cha của nó.

Nội dung

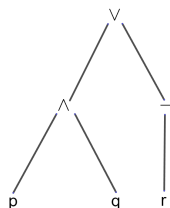
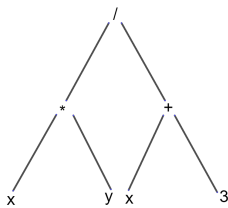
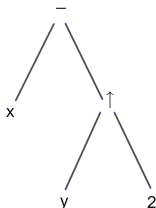
- 1 Duyệt cây
 - Hệ địa chỉ phổ dụng
 - Các thuật toán duyệt cây
- 2 Ký pháp trung tố, tiền tố, và hậu tố
- 3 Cây khung
 - Mở đầu
 - Tìm kiếm ưu tiên chiều sâu
 - Tìm kiếm ưu tiên chiều rộng

Cây biểu thức

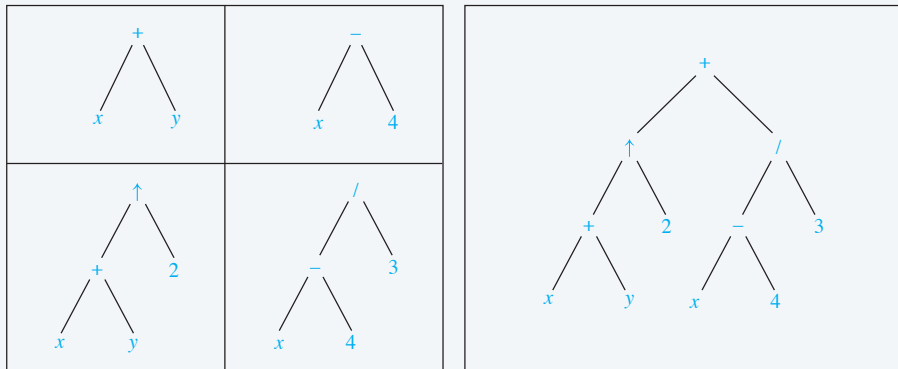
Định nghĩa

Cây biểu thức là cây nhị phân biểu diễn một biểu thức toán học nào đó, trong đó các đỉnh trong biểu thị các phép toán, các lá biểu thị các số hay các biến, mỗi phép toán tác động lên cây con bên trái và cây con bên phải (theo thứ tự Cây con trái-Phép toán- Cây con phải) hoặc tác động lên cây con duy nhất.

Ví dụ: Biểu thức $x - y^2$, $(x * y) / (x + 3)$, $(p \wedge q) \vee \neg r$ có cây biểu thức là:



Cây biểu thức



Hình: Cây nhị phân biểu diễn $((x + y) \uparrow 2) + ((x - 4) / 3)$

Khi duyệt cây trên theo kiểu trung thứ tự, ta được dạng trung tố của cây biểu thức là $x + y \uparrow 2 + x - 4 / 3$.

Nhận xét:

Tính nhập nhằng của dạng trung tố

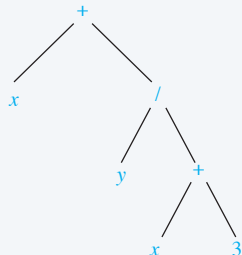
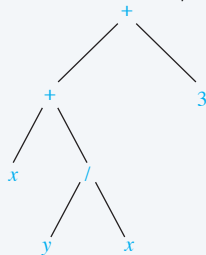
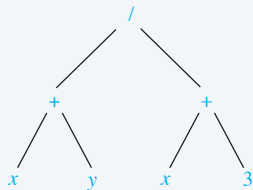
Định nghĩa

Dạng trung tố hay ký pháp trung tố của một biểu thức là dãy các ký tự (của biểu thức đó) khi duyệt cây biểu thức của biểu thức đó theo kiểu trung tố.

Các cây có gốc sau biểu diễn biểu thức:

$(x + y)/(x + 3)$, $(x + (y/x)) + 3$, và $x + (y/(x + 3))$

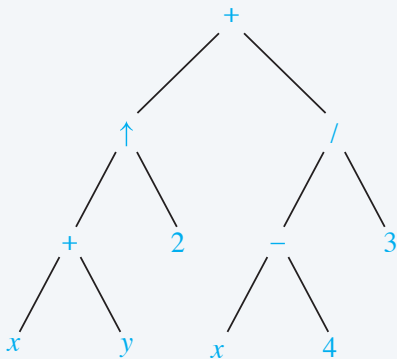
Các cây này đều có biểu thức trung tố $x + y/x + 3$



Dạng tiền tố của biểu thức - ký pháp Ba Lan

Định nghĩa

Dạng tiền tố hay ký pháp trung tố của một biểu thức là dãy các ký tự (của biểu thức đó) khi duyệt cây biểu thức của biểu thức đó theo kiểu tiền tố.



Dạng tiền tố của biểu thức

$$((x + y) \uparrow 2) + ((x - 4) / 3)$$

là

$$+ \uparrow + x y 2 / - x 4 3$$

Tính giá trị của biểu thức tiền tố

$$+ \quad - \quad * \quad 2 \quad 3 \quad 5 \quad / \quad \uparrow \quad 2 \quad 3 \quad 4$$

$$\underbrace{\quad \quad \quad}_{2 \uparrow 3 = 8}$$

$$+ \quad - \quad * \quad 2 \quad 3 \quad 5 \quad / \quad 8 \quad 4$$

$$\underbrace{\quad \quad}_{8 / 4 = 2}$$

$$+ \quad - \quad * \quad 2 \quad 3 \quad 5 \quad 2$$

$$\underbrace{\quad \quad}_{2 * 3 = 6}$$

$$+ \quad - \quad 6 \quad 5 \quad 2$$

$$\underbrace{\quad \quad}_{6 - 5 = 1}$$

$$+ \quad 1 \quad 2$$

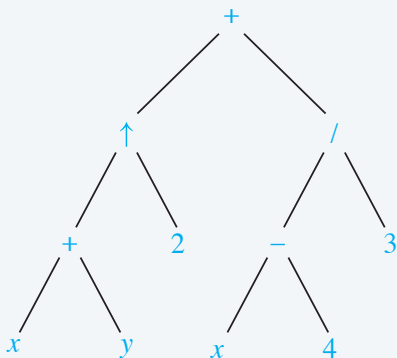
$$\underbrace{\quad \quad}_{1 + 2 = 3}$$

Value of expression: 3

Dạng hậu tố của biểu thức - Ký pháp Ba Lan ngược

Định nghĩa

Dạng hậu tố hay ký pháp trung tố của một biểu thức là dãy các ký tự (của biểu thức đó) khi duyệt cây biểu thức của biểu thức đó theo kiểu hậu tố.



Dạng hậu tố của biểu thức

$$((x + y) \uparrow 2) + ((x - 4)/3)$$

là

$$x y + 2 \uparrow x 4 - 3 / +$$

Tính giá trị của biểu thức hậu tố

7 2 3 * - 4 ↑ 9 3 / +

└──────────┘

$$2 * 3 = 6$$

7 6 - 4 ↑ 9 3 / +

└──────────┘

$$7 - 6 = 1$$

1 4 ↑ 9 3 / +

└──────────┘

$$1^4 = 1$$

1 9 3 / +

└──────────┘

$$9 / 3 = 3$$

1 3 +

└──────────┘

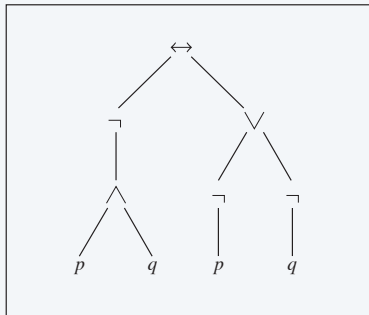
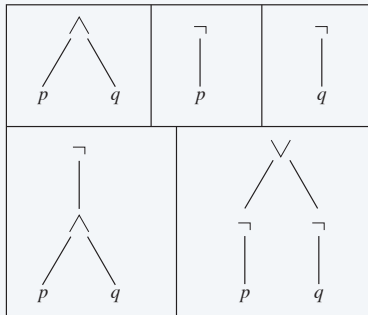
$$1 + 3 = 4$$

Value of expression: 4

Cây có gốc biểu diễn mệnh đề

- Tìm cây có gốc biểu diễn cho mệnh đề phức hợp

$$(\neg(p \wedge q)) \leftrightarrow (\neg p \vee \neg q)$$



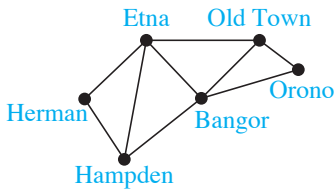
- Dạng tiền tố, trung tố, và hậu tố của biểu thức này là gì?

Nội dung

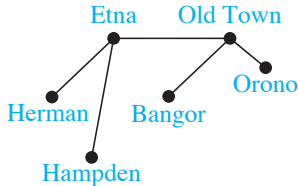
- 1 Duyệt cây
 - Hệ địa chỉ phổ dụng
 - Các thuật toán duyệt cây
- 2 Ký pháp trung tố, tiền tố, và hậu tố
- 3 Cây khung
 - Mở đầu
 - Tìm kiếm ưu tiên chiều sâu
 - Tìm kiếm ưu tiên chiều rộng

Bài toán

- Hệ thống giao thông của Maine được mô tả như dưới đây.
- Cách duy nhất để đi lại giữa những con đường trong mùa đông là phải cào tuyết thường xuyên.
- Tìm cách cào tuyết một số **ít nhất** các con đường sao cho luôn luôn có đường thông suốt nối hai thành phố bất kỳ.



(a)

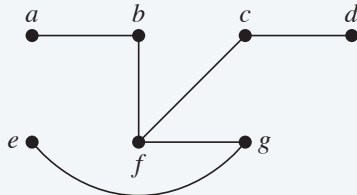
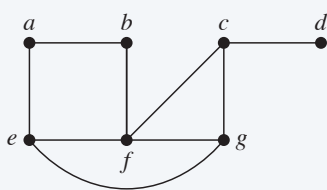


(b)

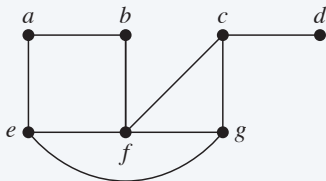
Cây khung

Định nghĩa

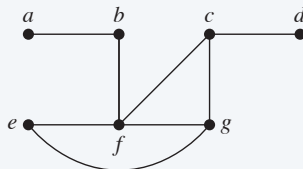
Cho G là một đồ thị đơn. Một cây được gọi là *cây khung* của G nếu nó là một đồ thị con của G và chứa mọi đỉnh của G .



Tìm cây khung bằng cách xóa cạnh

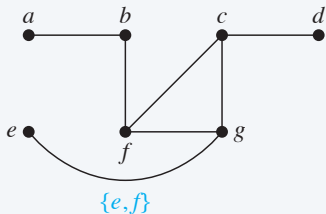


→

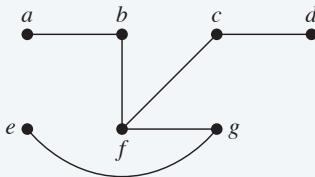


Edge removed: $\{a, e\}$

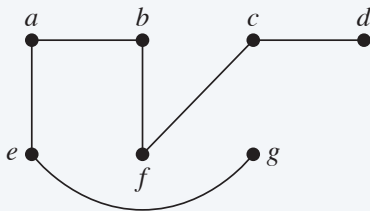
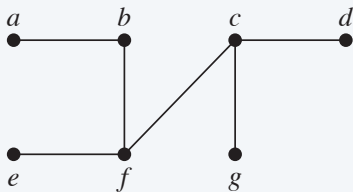
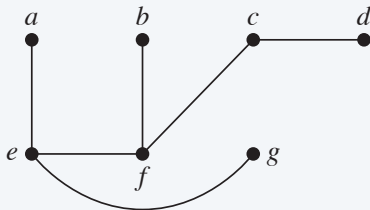
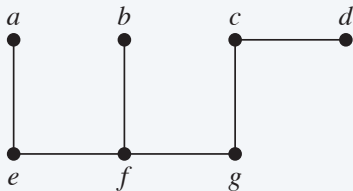
→



→



Một đồ thị có thể có nhiều cây khung



Hình: Một vài cây khung có của đồ thị trước

Định lý

Một đơn đồ thị là liên thông nếu và chỉ nếu nó có cây khung.

Chứng minh: Thật vậy, nếu đồ thị G có cây khung T , do T là đồ thị con chứa tất cả các đỉnh của G mà T liên thông nên G liên thông. Ngược lại, nếu G liên thông và G có chu trình thì ta xóa 1 cạnh trong chu trình đó, ta được G_1 vẫn liên thông, nếu G_1 vẫn có chu trình thì ta lại xóa 1 cạnh của chu trình đó, ta được G_2 vẫn liên thông, ..., cứ tiếp tục như vậy nhưng do số cạnh là hữu hạn nên quá trình trên sẽ dừng khi được G_k , lúc này G_k không có chu trình, khi đó G_k chính là cây khung của G .

Phương pháp tìm cây khung của đồ thị

- Xuất phát từ một đỉnh tùy ý làm gốc
- Xây dựng đường đi từ gốc bằng cách ghép các cạnh vào sao cho mỗi cạnh được ghép sẽ nối một đỉnh trên đường đi với một đỉnh còn chưa thuộc đường đi.

Ứng dụng thuật toán DFS để tìm cây khung

procedure DFS(*Đồ thị liên thông* G)

T = cây chỉ chứa một đỉnh u của G ;

 visit (u);

procedure visit(*Đỉnh* v của G)

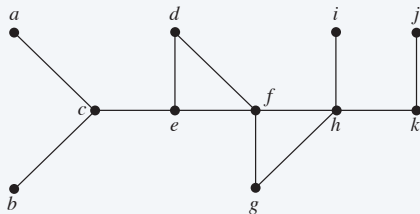
for mỗi đỉnh w liền kề với v và $w \notin T$ **do**

 thêm đỉnh w và cạnh $\{v, w\}$ vào cây T ;

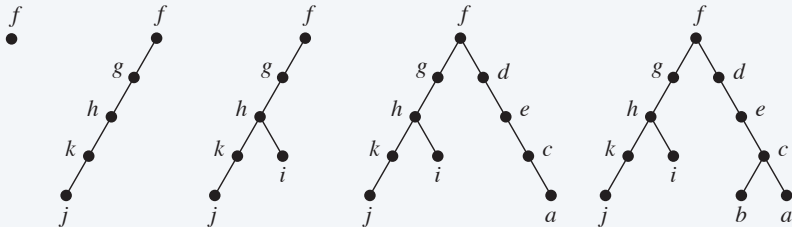
 visit (w);

end

Dùng DFS



Hãy dùng thuật toán DFS để tìm cây khung cho đồ thị trên.



Ứng dụng thuật toán BFS để tìm cây khung

procedure BFS(*Đồ thị liên thông* G)

T = cây chỉ chứa một đỉnh u của G

L = danh sách rỗng

 thêm đỉnh u vào danh sách L

while $L \neq \emptyset$ **do**

 xóa đỉnh đầu tiên v ra khỏi danh sách L

for *mỗi* đỉnh w liên kề với v **do**

if w chưa bao giờ nằm trong L và $w \notin T$ **then**

 thêm đỉnh w vào cuối danh sách L

 thêm đỉnh w và cạnh $\{v, w\}$ vào cây T

end

end

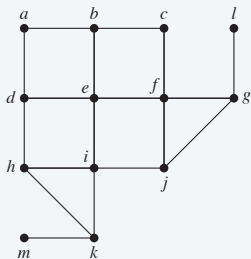
end

Thuật toán BFS-Giả mã

```

Procedure BFS(G)  % G có các đỉnh là  $a_1, a_2, \dots, a_n$  và tập cạnh  $E$ .
 $u = a_1$ ,  $KeTiep(a_i) = 0, \forall i$ ,  $T = \{u\}$  % b là đỉnh cuối ở danh sách  $L$ .
Procedure VS(u)
if  $KeTiep(u) = 0$  then  $i = 2$ 
  While  $i \leq n$  do
    if  $a_i \notin T$  and  $(u, a_i) \in E$  then
       $KeTiep(u) = a_i$ ,  $j := i + 1$ ,  $i = n$ ,  $T = T \cup \{a_i\}$ ,
       $E_T = E_T \cup \{(u, a_i)\}$ ,  $b = a_i$ 
    else  $i := i + 1$ .
  For  $k = j$  to  $n$  do
    if  $a_k \notin T$  and  $(u, a_k) \in E$  then
       $KeTiep(b) = a_k$ ,  $b = a_k$ ,  $T = T \cup \{a_k\}$ ,
       $E_T = E_T \cup \{(u, a_k)\}$ .
  if  $KeTiep(u) \neq 0$  then VS( $KeTiep(u)$ )
else For  $k = 1$  to  $n$  do if  $a_k \notin T$  and  $(u, a_k) \in E$  then
   $KeTiep(b) = a_k$ ,  $b = a_k$ ,  $T = T \cup \{a_i\}$ ,  $E_T = E_T \cup \{(u, a_k)\}$ .
  VS( $KeTiep(u)$ ).

```



Hãy dùng thuật toán BFS bắt đầu từ e để tìm cây khung cho đồ thị trên.

e

